



Conversation input

Compose a message with deployment, attachment and voice controls.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-conversation-input



When this library is useful

Compose a message with deployment, attachment and voice controls.

01

Send a message

Collect text and attachments, then invoke a host callback.

02

Control generation

Swap sending for a stop action while streaming.

03

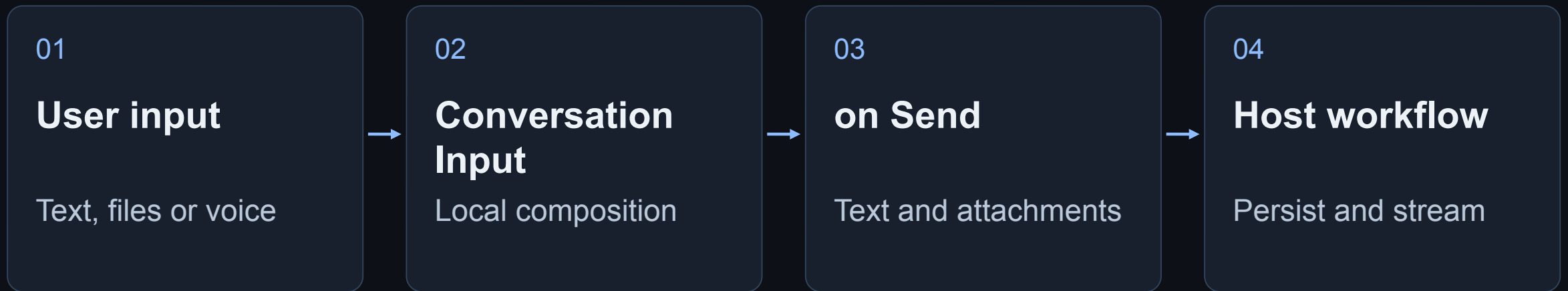
Support rich input

Wire deployment selection, settings and audio transcription.



Follow the ownership boundary

Inputs and effects stay explicit.



Library: Composer interaction, local text/attachment state and input controls.



The public contract to start with

ConversationInput · selected surface; full API index is in the bundle.

Input or operation	Contract
onSend (optional)	((message: string, attachments: Attachment[]) => void)
onUploadAttachment (optional)	((attachment: Attachment) => Promise<UploadedAttachmentResult>)
isStreaming (optional)	boolean
onStop (optional)	(() => void)



A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
function ComposerDemo() {  
  const [sent, setSent] = useState('');  
  return <>  
    <ConversationInput placeholder="Write a demo message"  
      onSend={ (message) => setSent(message)} />  
    <p role="status">{sent ? `Last demo message: ${sent}` : 'Ready'}</p>  
  </>;  
}
```

Full example: [examples/conversation-input.tsx](#) · uses [@epam/ai-dial-conversation-input](#)



Wire a realistic host workflow

Keep conversation input focused on its public responsibility.

01

Collect intent

Let the composer gather text and emit onSend.

02

Start the request

The host chooses the conversation and calls its adapter.

03

Reflect generation

Pass the current input and generation state back to the UI.



Available customization

01

Controlled updates

messageRevision and
textInsertion coordinate
host-driven edits.

02

Separate attachment UI

Import attachment cards
and trays from
attachment-input.

03

Voice extension

Inject a transcription
callback and supported
deployment data.



Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

Evidence	Where to look
Real consumer	chat/hooks/conversation/useAudioTranscription.ts
Test evidence	uses default label when no deployment is selected
Test evidence	transcribes actual browser MIME and releases microphone before recognition completes



Constraints that affect integration

01

No completion client

onSend is the integration point for a real generation flow.

02

Provider assumptions

Feature-specific settings and menus need caller-supplied data.

03

Avoid state resets

Keep initialization and host insertion changes intentional.